

Executive Summary of Rust Ownership Pedagogy for Systematic Metagenomics

Computer Science 131
University of California, Los Angeles

Abstract

Systematic Metagenomics has written the code for most of its CPU and GPU cycles in low-level languages such as C/C++/CUDA, which are not memory-safe. Because of this, the program's behavior is unreliable and prone to crashing, especially for large or complicated runs. Rust offers a compelling alternative to these languages: its ownership type system and compile-time checks can prevent developers from writing code that results in undefined behavior. Rewriting all of the C/C++/CUDA code in Rust would thus make it more reliable. However, this improvement comes at the tradeoff of Rust being more difficult to learn and use, with the most significant barrier to learning being the very same ownership type system that makes it so reliable. The research paper *A Grounded Conceptual Model for Ownership Types in Rust* (2026) by Crichton, Gray and Krishnamurthi (referred to as CGK from this point forward) addresses this issue by devising a new conceptual model of Rust's ownership system to improve pedagogy. For Systematic Metagenomics' developers, who are already familiar with low-level coding languages, the CGK conceptual model can significantly enhance understanding of Rust, but should not be viewed as a replacement for the traditional method of learning Rust, the book *The Rust Programming Language* (referred to as TRPL from this point forward).

1 Background and Motivation

Low-level languages like C/C++/CUDA have the advantage of being close to hardware and allow programmers to directly interact with the computer's memory. In these languages, developers have manually allocate all of the memory they intend to use and then manually free it when they are done using it. Forgetting to allocate memory correctly, accessing memory incorrectly or forgetting to free memory when done can all lead to some of the most common and frustrating software bugs such as dereferencing null pointers, segmentation faults, memory leaks, etc...

By contrast, high-level languages like Java and Python do not allow programmers to access memory directly, instead allocating memory on their behalf when needed and then deallocating it when it is no longer being used, in a process known as automated garbage collection. This has the advantage of preventing all the memory-related bugs in the low-level languages, but it also has the downside of being a lot less efficient. Garbage collection and memory allocation algorithms can have expensive and unnecessary overhead (in terms of both runtime and resources) and developers lose the freedom to manually optimize their code for memory efficiency.

Rust is a relatively modern language that was invented to be the solution to this conundrum - "the best of both worlds". It's low-level enough to allow programmers to access memory directly while also preventing many of the bugs that make low-level languages a pain to use. Rust accomplishes this via its ownership type system, which uses static checks to prevent these errors. This ownership system is conceptually unique and vastly different from the type systems that exist in most other programming languages, which is a huge learning curve for programmers and likely the most significant barrier preventing Rust from widespread adoption. CGK desired to change this by developing new ways of teaching Rust's ownership model to improve Rust learning outcomes. [?].

2 The Rust Programming Language (TRPL)

This book is the most common and traditional way new developers learn Rust. It is well structured and gives an excellent tutorial on Rust, but its approach to pedagogy has flaws. It gives thorough explanations of Rust's syntax illustrated with example code and plenty of practical experience through exercises and projects. But, as pointed out by the CGK research paper, it fails to teach the deeper concepts and reasoning behind the design of Rust. The CGK researchers tested this by designing an ownership inventory - a set of questions designed to measure Rust learners' understanding of ownership - and found that most Rust learners were able to correctly identify code that would be rejected by the borrow checker (64% suc-

cess rate) but most were not able to fix it (31% success rate). Reading the book makes the reason for this clear: it lacks emphasis on the underlying mechanisms and design philosophy of Rust. For example, Chapter 4.2 introduces the concept that mutable references must be exclusive, otherwise the Rust compiler will produce a compilation error. It mentions that this is because data races would occur if the Rust compiler didn't produce a compilation error, but fails to produce a concrete example of a data race. This can lead to the deficiencies in the ownership inventory scores seen in the research paper.

3 The CGK Approach

The conceptual model is a good way to explain what the Rust compiler is doing but it is an abstraction, and like all abstractions and models, it has the weakness of simplifying the concept by leaving things out. It may lead to the misconception that Rust is using literal RWO values, which it is not.

3.1 The Conceptual Model

CGK invented a conceptual model that reframes the borrow checker as maintaining a *flow-sensitive permission state*: at each program point, every path into memory carries a set of permissions—**Read**, **Write**, or **Own** (RWO). The compiler refuses to compile code that would exercise a permission not currently held. This framing makes the borrow checker's reasoning much clearer:

- A move transfers O (and with it R and W) from one place to another.
- A shared borrow `&T` grants R to the borrower while temporarily removing W and O from the original.
- A mutable borrow `&mut T` grants R and W to the borrower while removing all permissions from the original for the borrow's duration.

3.2 Aquascope

CGK implemented *Aquascope*, a compiler plugin that renders the permission state at each program point as an annotated diagram alongside the source code. The visualization makes abstract permission state concrete, aiding developers' mental models.

3.3 Results

An A/B deployment against TRPL (N = 342) improved average Ownership Inventory scores from 48% to 57%. Comprehension questions targeting the model itself were answered correctly 72% of the time.

4 Strengths and Weaknesses

4.1 Strengths

Familiar mental model for most developers. RWO permissions map naturally onto Linux/Shell semantics: files also have to be given either read (R), write (W) or execute (X) permissions.

Empirical pedagogical basis. CGK started by designing a test to understand what misconceptions beginners actually had. This is more trustworthy than an intuition-driven approach.

Supports reading compiler diagnostics. The borrow checker's error messages reference concepts ("value does not live long enough," "cannot borrow as mutable because it is also borrowed as immutable") that map directly onto permission revocations in the CGK model, making the errors self-explanatory to a developer who knows the model.

4.2 Weaknesses

Model is an abstraction, not the implementation. RWO permissions are a pedagogical device; the compiler does not literally maintain an RWO table. Developers who mistake the model for the implementation may become confused.

Limited coverage of advanced features. CGK focuses on ownership and borrowing. It does not cover traits, trait objects, `unsafe`, or `Send/Sync`—all of which our GPU and multi-threaded code will require. TRPL remains essential for these topics.

Small effect in absolute terms. A 9-percentage-point improvement on the Inventory is statistically significant but leaves average scores at 57%. Learning Rust will still require sustained effort and practice.

5 Conclusion

I recommend that all developers migrating C/C++/CUDA code to Rust read both the standard TRPL tutorial in addition to the CGK research paper.

The CGK model is particularly valuable at API design time: before writing a new module, developers should sketch the permission flow of each function (what permissions does this borrow, for how long, does it transfer ownership?)

I do not recommend replacing TRPL with CGK; they are complementary. TRPL covers a lot of material that CGK does not.

Rust Version

All analysis assumes Rust stable v1.95.0 (the current stable release as of June 2026). The Aquascope visualizer may require a recent nightly toolchain; developers should consult the project repository for compatibility.

References

- [1] Potanin A. Technical perspective: A conceptual model reinterprets Rust’s borrow checker in more user-friendly terms. *Commun ACM*. 2026;69(5):104. <https://doi.org/10.1145/3796539>
- [2] Crichton W, Gray G, Krishnamurthi S. A grounded conceptual model for ownership types in Rust. *Commun ACM*. 2026;69(5):105–113. <https://doi.org/10.1145/3796537>
- [3] Klabnik S, Nichols C. *The Rust Programming Language*. 2nd ed. San Francisco, CA: No Starch Press; 2023.

A Experience with Coding Problems

To evaluate both approaches I attempted two coding problems: (1) a function with overlapping borrows analogous to the `reverse` pattern in CGK, and (2) a generic function requiring lifetime annotations analogous to `add_displayable`.

A.1 Problem 1: Overlapping Borrows

```
fn remove_val(v : mutVec < i32 >, val : i32) for i in 0..v.len() if v[i] == val { v.remove(i); } // ERROR : index out of bounds + borrow conflict
```

With TRPL alone I could see the borrow error but initially "fixed" it by cloning the vector—a correct but needlessly expensive approach.

After applying the CGK permissions model I asked: at the point of `v.remove(i)`, does `v` still hold R+W+O? Yes, but iterating with an index while mutating shifts the semantics. The model clarified that the correct idiomatic fix is to use `retain`:

```
fn remove_val(v : mutVec < i32 >, val : i32) { v.retain(|x| x != val); }
```

The permission-flow reasoning ("I need W+O on `v` for the whole call, not an ongoing immutable borrow from the iteration") was the direct path to the idiomatic solution.

A.2 Problem 2: Lifetime Annotations in a Generic API

This problem mirrors the genomic comparison use case: a function that takes a reference to a sequence and returns a slice of it.

```
fn first_un(seq : [u8]) -> [u8] if seq.is_empty() return []; let mut end = 1; while end < seq.len() { seq[end] == seq[0] { end += 1; } } seq[..end]
```

With TRPL I understood that the lifetime of the return value must be tied to `seq`, but the book's lifetime elision rules are presented as syntactic patterns to memorize rather than as semantic constraints. I initially added `'static` (as 12/16 participants in CGK's study did), which was rejected because `seq` is not `'static`.

The CGK model reframed the question: the returned `&[u8]` borrows from `seq`, so it cannot outlive `seq`'s borrow. Lifetime elision handles this automatically for single-input functions; the compiler accepted the function as written. The key insight was understanding *why* elision works (the output borrow is tied to the one input borrow), not just that it works.

For a more complex case with two input lifetimes:

```
fn longer_un<'a> (a : 'a [u8], b : 'b [u8]) -> 'a [u8] if a.len() >= b.len() else b
```

Here the CGK approach made the annotation natural: the returned reference borrows from whichever of `a` or `b` is chosen at runtime, so the caller must ensure both live as long as the return value—hence the single shared lifetime `'a`.

B AI Usage:

Turn Potanin A. Technical Perspective: A Conceptual Model Reinterprets Rust’s Borrow Checker in More User-Friendly Terms. *Commun ACM*. 2026;69(5):104. doi:[10.1145/3796539](https://doi.org/10.1145/3796539). Crichton W, Gray G, Krishnamurthi S. A Grounded Conceptual Model for Ownership Types in Rust. *Commun ACM*. 2026;69(5):105–113. doi:[10.1145/3796537](https://doi.org/10.1145/3796537). and The Rust Programming Language into a Bibliography in LaTeX10:11 PMClaude responded: This is a straightforward LaTeX bibliography task — no docx needed. Check docx skillCheck docx skillThis is a straightforward LaTeX bibliography task — no docx needed. Here’s the .bib file:Created a file, read a fileCreated a file, read a fileHere’s your references.bib with all three entries:

Potanin2026 — the Technical Perspective piece, @article with single-page 104 Crichton2026 — the main research article, @article with page range 105–113 (double dash for LaTeX en-dash) Klabnik2023 — The Rust Programming Language, @book pointing to the 2nd edition (No Starch Press, 2023); adjust edition/year if you’re citing the first or online edition

A few notes:

Braces around Rust, ACM, and A in titles prevent BibTeX from lowercasing them. Cite with [1], [2], and [3]. Works with both BibTeX and BibLaTeX; if you use BibLaTeX, you can swap journal → journaltitle. ReferencesBIB DownloadYou said: like a bibliography I can put into a LaTeX documentlike a bibliography I can put into a LaTeX document10:14 PMClaude responded: Paste this directly into your .Created a file, read a fileCreated a file, read a filePaste this directly into your .tex document where you want the references to appear. Cite with [1], [2], and [3]. You’ll also need url in your preamble for the DOI links to render properly.